How the Game Really Works: A Code Walkthrough

The Detective's Dossier. Team Code Detektives, Graphic Era University

16 October 2026

1. Launching the game
2. How the mystery is stored
3. The rules engine (Game.h)
4. The validator (Validator.h)
5. The science puzzles (Forensics.h)
6. Movement and collision (Player.h and TileMap.h)
7. Reading map files (JsonLite.h and TiledMap.h)
8. Interaction rules (Interact.h)
9. The screen loop and keys (GameWidget.cpp)
10. Cross-examination on screen
11. Drawing the world
12. The lab and dial screens
13. How the tests prove things
14. One action through every layer
15. One-page cheat sheet

What this document is. A guided tour of the most important code in your project. For each major part you get the real code (copied straight from your files, with file names and line numbers), then a step-by-step explanation, then a worked example with real numbers or a real game moment. Read the code first, try to explain it to yourself, then read the explanation.

How the parts connect (keep this picture in your head).

```
You press a key
   -> Qt calls GameWidget::keyPressEvent            (screen: GameWidget.cpp)
   -> the mode decides which handler runs            (Explore, Talk, Present, Lab, ...)
   -> the handler asks the engine                    (Game.h: move, collect, present, accuse)
   -> the engine checks the mystery data             (Case.h, Entities.h, CaseRastogi.h)
   -> the engine returns a Result / Verdict          (never draws, never prints)
   -> the screen shows a message or loads a room     (say(), loadLocation())
   -> 60 times a second paintEvent redraws the world (drawWorld, HUD, overlays)
```

Contents. 1 Launching the game. 2 How the mystery is stored. 3 The rules engine (Game.h). 4 The validator. 5 The science puzzles. 6 Movement and collision. 7 Reading map files. 8 Interaction rules. 9 The screen loop and keys. 10 Cross-examination on screen. 11 Drawing the world. 12 Lab and dial screens. 13 How the tests prove things. 14 One action through every layer. 15 One-page cheat sheet.

1. Launching the game

What it does: starts Qt, builds the whole mystery in memory, loads the pictures, loads the first room and starts a timer that drives everything.

main_gui.cpp, lines 6 to 17

```
        int main(int argc, char** argv) {
            QApplication app(argc, argv);
            try {
                GameWidget w;
                w.resize(960, 720);
                w.show();
                return app.exec();
            } catch (const std::exception& e) {
                QMessageBox::critical(nullptr, "Could not start", QString::fromUtf8(e.what()));
                return 1;
            }
        }
```

Walk through `main`:

1. `QApplication app(argc, argv)` starts Qt. It owns the event loop that delivers key presses, timer ticks and paint requests.
2. The whole game is built inside `try`. If anything goes wrong while loading (a missing picture, a broken map), the constructor throws, and the `catch` shows a message box instead of crashing silently.
3. `w.resize(960, 720); w.show();` opens the window. `app.exec()` then waits for events until the window closes.

GameWidget.cpp, lines 50 to 71

```
GameWidget::GameWidget(QWidget* parent)
    : QWidget(parent),
      case_(makeRastogiCase()),
      game_(std::make_unique<Game>(case_)),
      collision_(std::vector<std::string>{"#"}),
      tileset_(q(std::string(ASSET_DIR) + "/tilesets/mansion.png")),
      characters_(q(std::string(ASSET_DIR) + "/sprites/characters.png")),
      ink_(rastogiInkPuzzle()) {
    if (tileset_.isNull()) throw std::runtime_error("Cannot load tileset: " + std::string(ASSET_DIR) + "/tilesets/mansion.png");
    if (characters_.isNull()) throw std::runtime_error("Cannot load sprites: " + std::string(ASSET_DIR) + "/sprites/characters.png"
    for (const auto& kv : case_.suspects()) if (kv.second.accusable()) accusable_.push_back(kv.first);
    std::sort(accusable_.begin(), accusable_.end());

    setWindowTitle("The Detective's Dossier: The Old Way");
    setMinimumSize(800, 600);
    setFocusPolicy(Qt::StrongFocus);
    loadLocation(game_->location(), "");
    say("The Old Way", q(kIntro));
    connect(&timer_, &QTimer::timeout, this, [this] { tick(); });
    timer_.start(16);
    clock_.start();
}
```

Walk through the constructor, in the order it runs:

1. The part after the colon is the **initialiser list**. Members are built in the order they are declared in GameWidget.h, not the order written here. `case_(makeRastogiCase())` builds the entire mystery (7 rooms, 23 clues, 6 people, 15 statements). Then `game_(std::make_unique<Game>(case_))` builds the referee that points at that case. This is why `case_` must be declared before `game_` in the header.
2. `collision_(std::vector<std::string>{"#"})` is a placeholder one-tile wall map. `TileMap` has no default constructor (it throws on an empty map), so it needs *something* now; the real map replaces it in `loadLocation`.
3. `tileset_(...)` and `characters_(...)` load the two pictures from `ASSET_DIR`. `ASSET_DIR` is a macro that CMake sets to the project's assets folder.
4. Two `if (...isNull()) throw` checks turn a missing picture into a clear error message.
5. The loop over `case_.suspects()` fills `accusable_` with everyone except the witness (`accusable()` is false only for Dr. Reddy). `std::sort` puts them in alphabetical order so the accusation menu is stable.
6. `setFocusPolicy(Qt::StrongFocus)` lets the widget receive key presses.
7. `loadLocation(game_->location(), "")` loads the start room (the hall). `say(...)` shows the intro text.
8. `connect(&timer_, &QTimer::timeout, this, [this]{ tick(); })` says "every time the timer fires, call `tick()`". `timer_.start(16)` fires it every 16 ms, about 62 times a second. It starts last, after everything exists.

Worked example: from double-click to first frame. Qt starts, `makeRastogiCase()` runs (a few hundred small `add` calls), two pictures load, `hall.json` is read and parsed, the player is placed on the `start` spawn point, the intro message appears, and 16 ms later `tick()` runs for the first time and asks Qt to repaint.

2. How the mystery is stored

What it does: describes every room, clue, person and statement as plain objects with string ids that point at each other.

Entities.h, lines 9 to 23

```
class Entity {
public:
    Entity(Id id, std::string name, std::string description)
        : id_(std::move(id)), name_(std::move(name)), description_(std::move(description)) {}
    virtual ~Entity() = default;
    virtual std::string kind() const = 0;  // polymorphism: UI/DB can treat all entities uniformly

    const Id& id() const { return id_; }
    const std::string& name() const { return name_; }
    const std::string& description() const { return description_; }

private:
    Id id_;
    std::string name_, description_;
};
```

Entities.h, lines 27 to 39

```
class Location : public Entity {
public:
    Location(Id id, std::string name, std::string desc, std::vector<Id> exits, Id requiredClue = "")
        : Entity(std::move(id), std::move(name), std::move(desc)),
          exits_(std::move(exits)), requiredClue_(std::move(requiredClue)) {}
    std::string kind() const override { return "Location"; }
    const std::vector<Id>& exits() const { return exits_; }
    const Id& requiredClue() const { return requiredClue_; }

private:
    std::vector<Id> exits_;
    Id requiredClue_;
};
```

Entities.h, lines 43 to 63

```
class Clue : public Entity {
public:
    // A clue that is NOT lying around in a room: it is earned (a confession, a lab result, a doctor's finding).
    Clue& grantedOnly() { granted_ = true; return *this; }
    bool granted() const { return granted_; }
    Clue(Id id, std::string name, std::string desc, Id location,
         std::vector<Id> prerequisites = {}, Id contradicts = "")
        : Entity(std::move(id), std::move(name), std::move(desc)),
          location_(std::move(location)), prerequisites_(std::move(prerequisites)),
          contradicts_(std::move(contradicts)) {}
    std::string kind() const override { return "Clue"; }
    const Id& location() const { return location_; }
    const std::vector<Id>& prerequisites() const { return prerequisites_; }
    const Id& contradicts() const { return contradicts_; }

private:
    Id location_;
    std::vector<Id> prerequisites_;
    Id contradicts_;
    bool granted_ = false;
};
```

Entities.h, lines 81 to 90

```
        struct Statement {
            std::string text;                // what they say
            std::string press;               // extra detail when the player presses them
            Id pressReveals;                 // clue earned by pressing ("" = none)
            std::vector<Id> contradictedBy;  // clues that prove this false (empty = it is true)
            std::string confession;          // how they react when caught
            Id reveals;                      // clue earned when caught
            bool incriminating = false;      // true only for lies that point at the killer
            bool isLie() const { return !contradictedBy.empty(); }
        };
```

Walk through the classes:

- `Entity` is the abstract base: id, name, description, plus `kind()` which children must implement. The `= 0` makes it impossible to create a bare Entity.
- `Location` adds the **graph edges** (`exits_`, a list of ids of neighbouring rooms) and the **lock** (`requiredClue_`, the id of the clue needed to enter; empty means unlocked). The default argument `Id requiredClue = ""` means most rooms are built without a lock.
- `Clue` adds the room it lies in (`location_`), the **DAG edges** (`prerequisites_`, clues you must already hold), and `granted_`. `grantedOnly()` returns `*this` so you can chain it when building: `Clue(...).grantedOnly()`. A granted clue is earned from a conversation or lab, never picked up.
- `Statement` is the heart of cross-examination. `contradictedBy` lists clue ids that prove the statement false; **an empty list means the statement is true**, which is exactly what `isLie()` tests. `reveals` is the clue you earn when you catch it, and `incriminating` marks lies that point at the killer.

Case.h, lines 14 to 18

```
        void add(Location l) { locations_.emplace(l.id(), std::move(l)); }
        void add(Clue c)     { clues_.emplace(c.id(), std::move(c)); }
        void add(Suspect s)  { suspects_.emplace(s.id(), std::move(s)); }
        void addStatement(const Id& suspect, Statement s) { statements_[suspect].push_back(std::move(s)); }
        void addLab(LabTask t) { labs_.push_back(std::move(t)); }
```

Case.h, lines 42 to 46

```
        template <class M>
        static const typename M::mapped_type* find(const M& m, const Id& id) {
            auto it = m.find(id);
            return it == m.end() ? nullptr : &it->second;
        }
```

Walk through `Case`: `add(...)` is overloaded three times (Location, Clue, Suspect) and stores each object in its own `unordered_map` keyed by id. `find` is a template: given any of those maps and an id, it returns a pointer to the stored object, or `nullptr` if the id does not exist. The public getters `location(id)`, `clue(id)` and `suspect(id)` are one-line calls to it. That is how the whole program turns the string `"c_scene"` into the actual Clue in O(1).

How the real case is built. These are real lines from `CaseRastogi.h`:

CaseRastogi.h, lines 28 to 28

```
        c.add(Location("attic", "The Attic", "Dust, tin trunks, faded portraits. A cracked brass nameplate lies against a crate.", {"hall"}
"c_shanta_testimony"));
```

CaseRastogi.h, lines 37 to 37

```
        c.add(Clue("c_milk_jug", "The kitchen milk jug", "The kesar milk left in the jug. Dr. Reddy's strip test is NEGATIVE. The sleeping
went into the glass AFTER the milk left the kitchen.", "kitchen", {"c_milk_glass"}));
```

CaseRastogi.h, lines 94 to 96

```
        c.addStatement("s_rohit", {"After dinner I went to my room and stayed there, with a sleeping pill, until morning.",
            "I sleep badly. A pill is the only way.", "", {"c_mill_report"},
            "Fine. I went to him at half past ten and begged him not to change the will. He was calm, and alive. He said Raghav was coming
eleven and we would talk after.", "c_rohit_testimony", false});
```

- The attic line: id `attic`, one exit (`hall`), and the last argument `"c_shanta_testimony"` is the lock. You cannot enter until you hold Shanta's confession.
- The milk jug line: it lies in the kitchen and has the prerequisite `c_milk_glass` (a DAG edge). Without the doctor's glass finding, picking it up

gives `MissingPrereq`.

- Rohit's statement, field by field: what he says; what he adds when pressed; no clue earned by pressing (empty); contradicted by `c_mill_report`; his confession text; the clue earned when caught, `c_rohit_testimony`; and `false` for incriminating (his lie is shameful, not murderous).

Worked example. "What do I need to catch Rohit?" Find his statement 0, read `contradictedBy`: only `c_mill_report`. "Where is it?" `clue("c_mill_report")->location()` is the library. "What do I earn?" `c_rohit_testimony`.

3. The rules engine (Game.h)

What it does: `Game` remembers what the player has done and decides, for every action, whether it is allowed. It returns a reason and never draws anything. Six private members hold all the state: `loc_` (current room), `held_` (clues as a fast set), `found_` (clues in order found), `history_` (undo stack), `exposed_` and `pressed_` (caught or pressed statements, keyed like `s_raghav#1`), and `credibility_` (starts at 6).

3.1 Moving between rooms

Game.h, lines 27 to 38

```
    Result move(const Id& to) {
        const Location* here = case_.location(loc_);
        const Location* dest = case_.location(to);
        if (!dest) return Result::Unknown;
        bool adjacent = false;
        for (auto& e : here->exits()) if (e == to) adjacent = true;
        if (!adjacent) return Result::NotAdjacent;
        if (!dest->requiredClue().empty() && !has(dest->requiredClue())) return Result::Locked;
        history_.push({Action::Move, loc_});
        loc_ = to;
        return Result::Ok;
    }
```

Walk through `move`:

1. `case_.location(...)` looks up the current room and the destination (hash lookups, O(1)). A missing destination returns `Unknown`.
2. The `for` loop scans the current room's exit list. If the destination is not in it, the rooms are not connected: `NotAdjacent`.
3. If the destination has a `requiredClue` and you do not hold it, the door is `Locked`. The lock lives on the destination room, so it applies no matter which door you use.
4. Only now, after every check has passed, does it change anything: it pushes the room you are *leaving* on the undo stack, then sets `loc_`. A failed move therefore never pollutes the undo history.

Worked example. You are in the hall and call `move("attic")` without Shanta's confession. Step 1 passes, step 2 passes (the hall lists the attic as an exit), step 3 finds `requiredClue = "c_shanta_testimony"` and `has(...)` is false, so you get `Locked`. Nothing changed. After you catch Shanta, the same call returns `Ok`, pushes `{Move, "hall"}` and sets `loc_ = "attic"`.

3.2 Picking things up: `collect`, `grant` and the private `add`

Game.h, lines 41 to 48

```
    Result collect(const Id& clueId) {
        const Clue* k = case_.clue(clueId);
        if (!k) return Result::Unknown;
        if (has(clueId)) return Result::AlreadyHave;
        if (k->granted() || k->location() != loc_) return Result::WrongPlace;
        if (!prerequisitesMet(*k, held_)) return Result::MissingPrereq;
        return add(clueId);
    }
```

Game.h, lines 51 to 57

```
    Result grant(const Id& clueId) {
        const Clue* k = case_.clue(clueId);
        if (!k) return Result::Unknown;
        if (has(clueId)) return Result::AlreadyHave;
        if (!prerequisitesMet(*k, held_)) return Result::MissingPrereq;
        return add(clueId);
    }
```

Game.h, lines 159 to 164

```
        Result add(const Id& clueId) {
            held_.insert(clueId);
            found_.push_back(clueId);
            history_.push({Action::Collect, clueId});
            return Result::Ok;
        }
```

Walk through:

- `collect` is for things lying in a room. It checks in order: the clue exists (`Unknown`); you do not already have it (`AlreadyHave`); it is not an earned-only clue and it is in *this* room (`WrongPlace`); and its prerequisites are held (`MissingPrereq`). Then it calls `add`.
- `grant` is for earned clues (confessions, lab results). It skips the room checks, but still checks existence, duplicates and prerequisites.
- `add` is private, so it is the only door into the dossier. It puts the id in the set `held_`, appends it to the vector `found_`, and pushes a `Collect` action on the undo stack.
- `prerequisitesMet(...)` (in Validator.h) loops over the clue's prerequisite ids and returns false if any is missing from `held_`. This single function is what turns the clue DAG into actual gameplay rules.

Worked example. In the kitchen, `collect("c_milk_jug")`. The clue exists, you lack it, it is in the kitchen, but its prerequisite `c_milk_glass` is not held: `MissingPrereq`. After Dr. Reddy's second statement is pressed, `grant("c_milk_glass")` succeeds, and the same `collect` now returns `Ok`.

3.3 Undo

Game.h, lines 60 to 70

```
        Result undo() {
            if (history_.empty()) return Result::NothingToUndo;
            Action a = history_.top(); history_.pop();
            if (a.type == Action::Move) {
                loc_ = a.arg;
            } else {
                held_.erase(a.arg);
                found_.erase(std::remove(found_.begin(), found_.end(), a.arg), found_.end());
            }
            return Result::Ok;
        }
```

Walk through: if the stack is empty return `NothingToUndo`. Otherwise copy the top action and pop it (LIFO). A `Move` action stored the room you left, so setting `loc_ = a.arg` puts you back. A `Collect` action stored the clue id, so it is erased from the set and removed from the vector with the erase-remove idiom (`std::remove` shifts the matching id to the end, `erase` cuts it off). Remember the quirk: actions from cross-examination are not reversed, only the clue is removed.

3.4 Cross-examination: `press` and `present`

Game.h, lines 93 to 102

```
        PressOutcome press(const Id& suspect, size_t idx) {
            const auto& st = case_.statements(suspect);
            PressOutcome o;
            if (idx >= st.size()) return o;
            o.valid = true;
            o.text = st[idx].press.empty() ? "They have nothing more to add." : st[idx].press;
            pressed_.insert(key(suspect, idx));
            if (!st[idx].pressReveals.empty() && grant(st[idx].pressReveals) == Result::Ok) o.revealed = st[idx].pressReveals;
            return o;
        }
```

Game.h, lines 105 to 123

```
PresentOutcome present(const Id& suspect, size_t idx, const Id& clue) {
    const auto& st = case_.statements(suspect);
    PresentOutcome o;
    if (idx >= st.size()) return o;
    if (!has(clue)) { o.kind = Present::NotHeld; return o; }
    if (exposed(suspect, idx)) { o.kind = Present::AlreadyExposed; o.text = "You have already caught them on that."; return o; }
    const Statement& s = st[idx];
    if (std::find(s.contradictedBy.begin(), s.contradictedBy.end(), clue) != s.contradictedBy.end()) {
        exposed_.insert(key(suspect, idx));
        o.kind = Present::Exposed;
        o.text = s.confession;
        if (!s.reveals.empty() && grant(s.reveals) == Result::Ok) o.revealed = s.reveals;
        return o;
    }
    penalize(1);
    o.kind = Present::Wrong;
    o.text = "That does not contradict what they said.";
    return o;
}
```

Walk through `press`: a bad statement number returns an invalid outcome. Otherwise it fills `o.text` with the statement's `press` text (or a default), marks the statement as pressed, and if the statement has a `pressReveals` clue it calls `grant`. `o.revealed` is set only if the grant really succeeded, so the screen can say "New evidence". Pressing never costs credibility.

Walk through `present`, in decision order:

1. Bad statement number: `Invalid`.
2. You do not hold the clue you tried to present: `NotHeld`, no penalty (you cannot present what you lack).
3. That statement is already caught: `AlreadyExposed`.
4. `std::find` searches `contradictedBy` for the clue. If found: the key `"suspect#index"` goes into `exposed_`, the confession text goes back to the screen, and the `reveals` clue is granted. Result `Exposed`.
5. Otherwise the evidence does not fit: `penalize(1)` and `Wrong`.

Worked example (a true statement is never contradicted). Rohit's second statement has an empty `contradictedBy`. Presenting any clue against it reaches step 5, so it costs a heart. That is how the game punishes random guessing, and it is one of the tests in the test file.

3.5 The final accusation

Game.h, lines 149 to 154

```
Verdict accuse(const Id& suspect) const {
    if (!case_.suspect(suspect)) return Verdict::UnknownSuspect;
    if (suspect != case_.culprit()) return Verdict::WrongSuspect;
    if (!case_.statements(suspect).empty()) return exposedIncriminating(suspect) >= 2 ? Verdict::Solved : Verdict::InsufficientEvid
    return proofAgainst(suspect).empty() ? Verdict::InsufficientEvidence : Verdict::Solved;
}
```

Walk through: an unknown id gives `UnknownSuspect`. Anyone who is not the real culprit gives `WrongSuspect`. For the culprit, if they have statements (our case), you win only when at least two of their *incriminating* lies are exposed; otherwise `InsufficientEvidence`. The last line is the older rule used by the first two cases. `accuse` is `const`: it changes nothing. The *screen* applies the penalty: any result other than `Solved` costs 2 credibility, with identical wording for "wrong person" and "not enough proof" so the killer cannot be found by guessing.

3.6 Trace: the first minutes of a game

State after each call. `held` is the clue set, `history` is the undo stack (top at the right).

#	Call	Result	What changed
1	start		loc = hall, held = {}, credibility 6
2	`move("library")`	Ok	loc = library; history = [Move(hall)]
3	`press("s_reddy", 0)`	revealed `c_body`	`grant` passes (no prerequisites); held = {c_body}; history = [Move(hall), Collect(c_body)]
4	`collect("c_body")`	WrongPlace	Earned-only clues cannot be picked up; no change
5	`collect("c_scene")`	Ok	held = {c_body, c_scene}
6	`present("s_rohit", 0, "c_scene")`	Wrong	`c_scene` is not in Rohit's `contradictedBy`; credibility 5

7	`present("s_rohit", 0, "c_mill_report")`	NotHeld	You have not collected it yet; no penalty
8	`collect("c_mill_report")`	Ok	held now has 3 clues
9	`present("s_rohit", 0, "c_mill_report")`	Exposed	`exposed_` gets `"s_rohit#0"`; `c_rohit_testimony` granted; history grows
10	`undo()`	Ok	Removes `c_rohit_testimony` (top of stack), but `s_rohit#0` stays exposed (the known quirk)

4. The validator (Validator.h)

What it does: proves a mystery is well-formed and can always be solved, by checking references, by checking that clue prerequisites have no cycle, and by letting a simulated perfect player try to finish the game.

4.1 BFS over the rooms

Validator.h, lines 15 to 30

```
inline std::unordered_set<Id> reachableLocations(const Case& c, const std::unordered_set<Id>& held) {
    std::unordered_set<Id> seen{c.start()};
    std::queue<Id> q;
    q.push(c.start());
    while (!q.empty()) {
        Id cur = q.front(); q.pop();
        for (const Id& next : c.location(cur)->exits()) {
            if (seen.count(next)) continue;
            const Id& req = c.location(next)->requiredClue();
            if (!req.empty() && !held.count(req)) continue;  // locked door
            seen.insert(next);
            q.push(next);
        }
    }
    return seen;
}
```

Walk through:

1. `seen` starts containing the start room; the queue starts with it too.
2. Take the front room. For each of its exits that is not already seen, look at the exit room's lock.
3. If there is a lock and you do not hold that clue, skip it: you cannot go there yet.
4. Otherwise mark it seen and queue it, so its own exits get explored later.
5. When the queue is empty, `seen` is every room you can reach right now.

Worked example (nothing held). Queue [hall]. Hall adds library, kitchen, family_wing, blue_room; the attic is skipped (needs `c_shanta_testimony`). Library: hall already seen; passage skipped (needs `c_blueprint`). Kitchen and family_wing only lead back to the hall. Blue_room: passage skipped. Result: 5 rooms. Each room entered the queue at most once and each exit was examined once: O(V + E).

4.2 Kahn's algorithm: is the clue graph a DAG?

Validator.h, lines 69 to 83

```
        // 2. Clue prerequisites must form a DAG (Kahn's topological sort)
        std::unordered_map<Id, int> indeg;
        std::unordered_map<Id, std::vector<Id>> out;
        for (auto& [id, k] : c.clues()) {
            indeg[id] += 0;
            for (auto& p : k.prerequisites()) { out[p].push_back(id); indeg[id]++; }
        }
        std::queue<Id> q;
        for (auto& [id, d] : indeg) if (d == 0) q.push(id);
        size_t sorted = 0;
        while (!q.empty()) {
            Id cur = q.front(); q.pop(); sorted++;
            for (auto& nxt : out[cur]) if (--indeg[nxt] == 0) q.push(nxt);
        }
        if (sorted != c.clues().size()) { err("clue prerequisites contain a cycle"); return r; }
```

Walk through:

1. `indeg[id] += 0;` makes sure every clue has an entry, even one with no prerequisites. For each prerequisite `p` of clue `id`, add an arrow `p -> id` (`out[p].push_back(id)`) and increase `id`'s in-degree.
2. Push every clue whose in-degree is 0 (it waits for nothing).
3. Repeatedly take a clue off the queue and count it (`sorted++`). It is now "done", so every clue that waited on it has one fewer prerequisite: decrement; if that reaches 0, push it.
4. At the end, if the number counted is not the number of clues, some clues never became ready. That can only happen if they wait on each other in a loop, so the case is rejected.

Worked example. The chain nameplate, strongroom_open, forged_fd, new_will has in-degrees 0, 1, 1, 1. Process nameplate: strongroom_open drops to 0. Process it: forged_fd drops to 0. Process it: new_will drops to 0. All four counted. If forged_fd also needed new_will, neither would ever reach 0, `sorted` would be too small, and validation fails.

4.3 The perfect-player simulation

Validator.h, lines 87 to 112

```
        // 3. Simulate a perfect player: collect, question, test, repeat until nothing changes.
        std::unordered_set<Id> held;
        auto earn = [&](const Id& id) {
            if (id.empty() || held.count(id) || !prerequisitesMet(*c.clue(id), held)) return false;
            held.insert(id); return true;
        };
        for (bool changed = true; changed;) {
            changed = false;
            auto reach = reachableLocations(c, held);
            for (auto& [id, k] : c.clues())
                if (!k.granted() && !held.count(id) && reach.count(k.location()) && prerequisitesMet(k, held)) { held.insert(id); changed =
}
            for (auto& [sid, vec] : c.allStatements())
                for (const Statement& st : vec) {
                    if (earn(st.pressReveals)) changed = true;
                    bool caught = false;
                    for (const Id& k : st.contradictedBy) if (held.count(k)) caught = true;
                    if (caught && earn(st.reveals)) changed = true;
                }
            for (const LabTask& t : c.labs()) {
                bool ready = true;
                for (const Id& n : t.needs) if (!held.count(n)) ready = false;
                if (ready && earn(t.grants)) changed = true;
            }
        }
        for (auto& [id, k] : c.clues())
            if (!held.count(id)) err("clue can never be obtained: " + id);
```

Walk through:

- `held` starts empty. `earn(id)` adds a clue only if it is valid, not already held and its prerequisites are met.
- The `for (bool changed = true; changed;)` loop repeats sweeps until a sweep changes nothing. Each sweep: (a) run the BFS with the clues held *right now* to see which rooms are open; (b) pick up every clue that is not earn-only, lies in an open room and has its prerequisites met; (c)

for every statement, earn what pressing gives, and if any contradicting clue is held, earn what catching the lie gives; (d) for every lab, if all needed clues are held, earn the result.

- Afterwards every clue must be in `held`. Any that is not can never be obtained by any player, so it is reported.

Why it terminates. `held` only grows and can contain at most 23 clues, so at most 23 sweeps change anything.

Worked example of the first sweeps. Sweep 1: open rooms are hall, library, kitchen, family_wing, blue_room. It collects `c_dinner`, `c_scene`, `c_mill_report`, `c_cheque`, `c_betting`, `c_pen`; earns Dr. Reddy's `c_body` and `c_milk_glass`; catches Rohit, Kabir and Shanta (their contradicting clues are now held), earning their testimonies; and runs the time-of-death lab for `c_tod`. Sweep 2: Shanta's confession opens the attic, so `c_blueprint` and `c_nameplate` are collected; `c_milk_jug` and `c_wife_text` now have their prerequisites. Sweep 3: the blueprint opens the passage, so `c_cufflink`, the dial clue, then `c_forged_fd`, `c_diary`, `c_new_will` follow, and the ink lab becomes ready. A final sweep changes nothing, so the loop stops. (The exact order inside a sweep can differ, because the clue map is unordered, but the final answer cannot.)

4.4 The cross-examination checks

Validator.h, lines 124 to 137

```
        // 4. (cross-examination cases) every lie can be exposed, the killer has >= 2 exposable incriminating lies,
        //    and nobody else has any incriminating statement.
        int killerExposable = 0;
        for (auto& [sid, vec] : c.allStatements())
            for (size_t i = 0; i < vec.size(); ++i) {
                const Statement& st = vec[i];
                bool exposable = false;
                for (const Id& k : st.contradictedBy) if (held.count(k)) exposable = true;
                if (st.isLie() && !exposable) err(sid + ": lie #" + std::to_string(i) + " can never be exposed");
                if (st.incriminating && sid != c.culprit()) err(sid + ": innocent person has an incriminating statement");
                if (st.incriminating && sid == c.culprit() && exposable) ++killerExposable;
            }
        if (killerExposable < 2) err("the killer has fewer than two exposable incriminating lies");
        if (c.suspect(c.culprit()) && !c.suspect(c.culprit())->accusable()) err("the culprit cannot be accused");
```

Walk through: for every statement of every person, `exposable` is true if at least one contradicting clue is obtainable. A lie that is never exposable is an error. An incriminating statement by anyone other than the culprit is an error (an innocent must not be framed by the data). For the culprit, count exposable incriminating lies; fewer than 2 is an error, because the accusation rule needs two. Finally the culprit must not be the witness.

5. The science puzzles (Forensics.h)

What it does: the two lab puzzles are pure maths with no Qt, so they can be unit-tested. The screen only shows them.

Forensics.h, lines 11 to 28

```
        struct TodPuzzle {
            double bodyTempC = 29.0;      // measured by the doctor
            double normalTempC = 37.0;
            double coolingPerHour = 1.0;
            int examMinute = 7 * 60 + 15; // minutes after midnight when it was measured (7:15 am)
            int toleranceMin = 45;        // answers within 45 minutes count
        };

        inline int todAnswerMinute(const TodPuzzle& p) {
            const double hours = (p.normalTempC - p.bodyTempC) / p.coolingPerHour;
            int m = (p.examMinute - static_cast<int>(std::lround(hours * 60))) % 1440;
            return m < 0 ? m + 1440 : m;
        }
        inline bool todCorrect(const TodPuzzle& p, int guessMinute) {
            int d = std::abs(guessMinute - todAnswerMinute(p));
            d = std::min(d, 1440 - d);
            return d <= p.toleranceMin;
        }
```

Walk through the time-of-death maths:

- Hours since death = (normal temperature minus body temperature) divided by the cooling rate = (37 - 29) / 1 = 8.
- Convert to minutes (`hours * 60`, rounded with `std::lround`) = 480 and subtract from the time of measurement, 7:15 am = 435 minutes after midnight: 435 - 480 = -45.
- `% 1440` keeps the answer inside one day, but in C++ a negative number stays negative, so `if (m < 0) m + 1440` fixes it: 1395 = 23:15.
- `todCorrect` measures how far the guess is from the answer on a 24-hour *circle*: `d = abs(guess - answer)` then `min(d, 1440 - d)`, so 23:50

and 00:10 count as 20 minutes apart. Within 45 minutes is accepted.

Forensics.h, lines 32 to 48

```
struct InkSample {
    std::string label;
    double spotCm, frontCm;
    double rf() const { return spotCm / frontCm; }
};
struct InkPuzzle {
    InkSample forged;
    std::vector<InkSample> pens;
};

// Index of the single pen whose Rf matches the forged ink (within tolerance), or -1 if none / ambiguous.
inline int inkMatch(const InkPuzzle& p, double tol = 0.005) {
    int found = -1;
    for (size_t i = 0; i < p.pens.size(); ++i)
        if (std::fabs(p.pens[i].rf() - p.forged.rf()) <= tol) { if (found >= 0) return -1; found = static_cast<int>(i); }
    return found;
}
```

Walk through the ink maths: `rf()` is spot distance divided by solvent distance. `inkMatch` loops over the pens; any pen within 0.005 of the forged ink's Rf is a candidate. If a second pen also matches it returns -1 (ambiguous); if exactly one matches it returns its index. With the real data: forged 4.26 / 6.0 = 0.71, Dayanand 3.10 / 5.0 = 0.62, Rohit 3.90 / 6.0 = 0.65, Raghav 5.68 / 8.0 = 0.71. Only index 2 (Raghav) matches.

6. Movement and collision (Player.h and TileMap.h)

What it does: moves the player a little each frame, never lets the player's box overlap a wall, and lets the player slide along walls instead of sticking to them.

Player.h, lines 17 to 31

```
void move(float dirX, float dirY, float dt, const TileMap& map) {
    float len = std::sqrt(dirX * dirX + dirY * dirY);
    if (len > 0) { dirX /= len; dirY /= len; }  // diagonal is not faster
    const float ox = x, oy = y;
    float nx = x + dirX * speed * dt;
    if (!collides(nx, y, map)) x = nx;
    float ny = y + dirY * speed * dt;
    if (!collides(x, ny, map)) y = ny;
    moving = (std::fabs(x - ox) + std::fabs(y - oy)) > 0.001f;
    if (len > 0) {
        if (std::fabs(dirX) > std::fabs(dirY)) facing = dirX < 0 ? 1 : 2;
        else facing = dirY < 0 ? 3 : 0;
    }
    anim = moving ? anim + dt : 0.f;
}
```

Player.h, lines 33 to 37

```
bool collides(float cx, float cy, const TileMap& map) const {
    const float e = 0.01f;
    return map.blockedPx(cx - half, cy - half) || map.blockedPx(cx + half - e, cy - half) ||
           map.blockedPx(cx - half, cy + half - e) || map.blockedPx(cx + half - e, cy + half - e);
}
```

TileMap.h, lines 24 to 27

```
char at(int tx, int ty) const {
    if (tx < 0 || ty < 0 || tx >= width() || ty >= height()) return '#';
    return rows_[ty][tx];
}
```

TileMap.h, lines 33 to 36

```
    bool blockedPx(float px, float py) const {
        return blockedTile(static_cast<int>(std::floor(px / tileSize_)),
                           static_cast<int>(std::floor(py / tileSize_)));
    }
```

Walk through `move`:

1. `dirX, dirY` is the input direction (for example `1, 0` for right, `1, 1` for down-right). It is divided by its own length, so a diagonal has length 1 like a straight move. Without this you would walk 41 percent faster diagonally.
2. `ox, oy` remember the old position, only to decide later whether the player really moved.
3. **X first:** `nx = x + dirX * speed * dt`. If the box at `(nx, y)` does not collide, accept it. **Then Y:** the same with the new `x`. Because the two axes are tested separately, a move that is blocked on one axis still succeeds on the other. That is the sliding.
4. `moving` is true only if the position really changed, so pushing into a wall does not play the walk animation.
5. `facing` follows the dominant direction (0 down, 1 left, 2 right, 3 up, matching the sprite sheet rows). `anim` counts seconds spent walking and resets to 0 when stopped.

Walk through `collides`: the player is a square box of side 18 (`half = 9`). The function tests its four corners with `blockedPx`. The tiny `e = 0.01` is subtracted from the right and bottom edges so a box that exactly touches the next tile is not counted as inside it.

Walk through `TileMap`: `at(tx, ty)` returns `'#'` for any position outside the map, so the player can never walk off the edge. `blockedPx(px, py)` converts a pixel position to a tile with `floor(px / tileSize)` and asks whether that tile is a wall.

Worked example (speed). Speed 130 pixels per second and a frame of 0.016 s: 130 x 0.016 = 2.08 pixels per frame. Diagonally each axis moves 2.08 x 0.7071 = 1.47 pixels. A 32-pixel tile takes about 15 frames.

Worked example (stopping at a wall). A wall tile starts at x = 128 (tile 4 times 32). The player is at x = 118.5 moving right. The next position is 120.58, and the right edge would be 120.58 + 9 - 0.01 = 129.57, which is in tile `floor(129.57 / 32) = 4`, a wall, so the move is refused and x stays 118.5. The player can get as far as x just under 119, about 9 pixels from the wall line, because half the box is 9.

Worked example (sliding). Hold right and down while standing against a wall on your right. X is refused every frame, Y is accepted, so the player slides down the wall at 1.47 pixels per frame instead of stopping.

7. Reading map files (JsonLite.h and TiledMap.h)

What it does: turns a Tiled `.json` file into numbers the game can use: which tiles to draw, which tiles are walls, and where the clues, doors, people and dial are.

7.1 The JSON parser

JsonLite.h, lines 58 to 70

```
    Json value() {
        ws();
        char c = peek();
        if (i >= s.size()) fail("unexpected end of input");
        if (c == '{') return object();
        if (c == '[') return array();
        Json j;
        if (c == '"') { j.type = Type::String; j.str = string(); return j; }
        if (c == 't') { expect("true"); j.type = Type::Bool; j.b = true; return j; }
        if (c == 'f') { expect("false"); j.type = Type::Bool; j.b = false; return j; }
        if (c == 'n') { expect("null"); return j; }
        return number();
    }
```

JsonLite.h, lines 129 to 148

```
        Json object() {
            ++i;
            Json j; j.type = Type::Object;
            ws();
            if (peek() == '}') { ++i; return j; }
            while (true) {
                ws();
                if (peek() != '"') fail("expected string key");
                std::string key = string();
                ws();
                if (peek() != ':') fail("expected ':'");
                ++i;
                j.members.emplace_back(key, value());
                ws();
                if (peek() == ',') { ++i; continue; }
                if (peek() == '}') { ++i; break; }
                fail("expected ',' or '}'");
            }
            return j;
        }
```

JsonLite.h, lines 115 to 128

```
        Json array() {
            ++i;
            Json j; j.type = Type::Array;
            ws();
            if (peek() == ']') { ++i; return j; }
            while (true) {
                j.arr.push_back(value());
                ws();
                if (peek() == ',') { ++i; continue; }
                if (peek() == ']') { ++i; break; }
                fail("expected ',' or ']'");
            }
            return j;
        }
```

Walk through (recursive descent):

- `value()` skips spaces, looks at one character and decides what comes next: `{` means an object, `[` an array, `"` a string, `t`, `f`, `n` the words true, false, null, anything else a number.
- `object()` skips the `{`. If the next character is `}` it is empty. Otherwise it loops: read a quoted key, expect a `:`, call `value()` for the content, then expect either `,` (another member) or `}` (done). Anything else throws an error with the exact character position.
- `array()` is the same without keys: call `value()`, then `,` or `]`.
- They call each other: an object contains values, a value can be an array, an array contains values. The nesting of the file *is* the recursion. The result is a tree of `Json` nodes.

Worked example. For `{"w":24,"layers":[1,2]}`: `value()` sees `{` and calls `object()`; it reads key `w`, calls `value()` which parses the number 24; it sees `,` and reads key `layers`, calls `value()` which sees `[` and calls `array()`; that calls `value()` twice for 1 and 2 and sees `]`; back in `object()` it sees `}` and finishes. Reading `root["layers"][1].number()` later gives 2.

7.2 Turning the tree into a map

TiledMap.h, lines 70 to 80

```
        if (type == "tilelayer") {
            TileLayer tl;
            tl.name = L["name"].string();
            tl.visible = L["visible"].boolean(true);
            if (!L["data"].isArray())
                throw std::runtime_error("Layer '" + tl.name + "': set Map > Tile Layer Format to CSV in Tiled");
            for (size_t k = 0; k < L["data"].size(); ++k)
                tl.data.push_back(static_cast<unsigned>(L["data"][k].number()) & 0x1FFFFFFFu);  // strip flip flags
            if (tl.data.size() != static_cast<size_t>(m.width_ * m.height_))
                throw std::runtime_error("Layer '" + tl.name + "' has the wrong number of tiles");
            m.layers_.push_back(std::move(tl));
```

TiledMap.h, lines 121 to 130

```
    TileMap collisionMap() const {
        const TileLayer* c = collisionLayer();
        std::vector<std::string> rows;
        for (int ty = 0; ty < height_; ++ty) {
            std::string row;
            for (int tx = 0; tx < width_; ++tx) row += c->data[ty * width_ + tx] ? '#' : '.';
            rows.push_back(row);
        }
        return TileMap(rows, tileW_);
    }
```

TiledMap.h, lines 138 to 159

```
    static MapObject readObject(const Json& o) {
        MapObject m;
        m.id = static_cast<int>(o["id"].number());
        m.name = o["name"].string();
        m.type = o["class"].string();                    // Tiled 1.9+
        if (m.type.empty()) m.type = o["type"].string(); // older Tiled
        m.x = static_cast<float>(o["x"].number());
        m.y = static_cast<float>(o["y"].number());
        m.w = static_cast<float>(o["width"].number());
        m.h = static_cast<float>(o["height"].number());
        const Json& props = o["properties"];
        for (size_t k = 0; k < props.size(); ++k) {
            const Json& p = props[k];
            const Json& v = p["value"];
            std::string val;
            if (v.type == Json::Type::String) val = v.str;
            else if (v.type == Json::Type::Bool) val = v.b ? "true" : "false";
            else if (v.type == Json::Type::Number) { std::ostringstream os; os << v.num; val = os.str(); }
            m.props[p["name"].string()] = val;
        }
        return m;
    }
```

Walk through the tile layer code: the layer's `data` must be an array (Tiled has to be set to save as CSV) or a helpful error is thrown. Each number is a tile id (a gid, 0 means empty). `& 0x1FFFFFFFu` clears the top three bits, because Tiled stores "flipped" and "rotated" flags there and we only want the plain tile number. The count must equal width times height, otherwise the file is corrupt.

Walk through `collisionMap`: for every cell of the layer named Collision, a non-zero tile becomes `'#'` (wall) and zero becomes `'.'` (floor). Those strings become the `TileMap` the player collides with. The Collision layer is never drawn.

Walk through `readObject`: reads the object's id, name, rectangle, and its type. Newer Tiled stores the type in `class`, older versions in `type`, so it tries both. Each custom property is converted to a string and stored in `props`, so `prop("clue_id")` returns the text or an empty string.

Illustrative example of one object in a map file (the numbers are made up; the properties match the real library map):

```
{"id": 7, "name": "Third bookcase", "class": "door", "x": 96, "y": 32, "width": 64, "height": 32,
 "properties": [{"name": "to", "value": "passage"}, {"name": "hidden_until", "value": "c_blueprint"}]}
```

After `readObject` this becomes a `MapObject` with `type = "door"`, `name = "Third bookcase"`, a 64 by 32 rectangle at (96, 32), and `props` containing `to = passage` and `hidden_until = c_blueprint`. The next part shows how those two properties are used.

8. Interaction rules (Interact.h)

What it does: decides which map objects the player can use right now, which one is nearest, and where the player stands after walking through a door.

Interact.h, lines 20 to 26

```
inline bool objectActive(const MapObject& o, const Game& g) {
    if (!isInteractive(o)) return false;
    const std::string hidden = o.prop("hidden_until");
    if (!hidden.empty() && !g.has(hidden)) return false;
    if ((o.type == "clue" || o.type == "dial") && g.has(o.prop("clue_id"))) return false;
    return true;
}
```

Interact.h, lines 28 to 32

```
inline float distanceToRect(float px, float py, const MapObject& o) {
    float dx = std::max(std::max(o.x - px, 0.f), px - (o.x + o.w));
    float dy = std::max(std::max(o.y - py, 0.f), py - (o.y + o.h));
    return std::sqrt(dx * dx + dy * dy);
}
```

Interact.h, lines 34 to 43

```
inline const MapObject* nearestInteractable(const TiledMap& m, const Game& g, float px, float py, float radius = 36.f) {
    const MapObject* best = nullptr;
    float bestD = radius + 0.001f;
    for (const auto& o : m.objects()) {
        if (!objectActive(o, g)) continue;
        float d = distanceToRect(px, py, o);
        if (d < bestD) { bestD = d; best = &o; }
    }
    return best;
}
```

Walk through `objectActive`: only clue, door, npc and dial objects can be used at all. If the object has `hidden_until` and you do not yet hold that clue, it is inactive. That is how the secret bookcase appears only after the blueprint. A clue or dial object whose clue you already hold also becomes inactive, which is why collected items stop glowing and stop being targets.

Walk through `distanceToRect`: the shortest distance from a point to a rectangle. `dx` is how far the point is outside the rectangle horizontally (0 if it is inside the x-range): the largest of “rectangle left minus point”, 0, and “point minus rectangle right”. `dy` is the same vertically. Then Pythagoras.

Worked example. Player at (50, 50), object rectangle x = 60, y = 40, w = 20, h = 20. `dx = max(max(60 - 50, 0), 50 - 80) = 10`. `dy = max(max(40 - 50, 0), 50 - 60) = 0`. Distance = 10, which is within the 36 pixel interaction radius.

Walk through `nearestInteractable`: loop over every object on the map, skip inactive ones, compute the distance, and remember the closest one that is within the radius. The loop starts with `bestD` just above the radius so anything farther away is ignored. If nothing qualifies it returns `nullptr` and the HUD shows no prompt.

Interact.h, lines 48 to 65

```
inline std::pair<float, float> arrivalPoint(const TiledMap& m, const TileMap& collision, const Id& from, Player probe) {
    const float ts = static_cast<float>(collision.tileSize());
    const MapObject* door = nullptr;
    if (!from.empty())
        for (const auto& o : m.objects()) if (o.type == "door" && o.prop("to") == from) { door = &o; break; }
    std::vector<std::pair<float, float>> spots;
    if (door) {
        const float cx = std::floor(door->cx() / ts) * ts + ts / 2, cy = std::floor(door->cy() / ts) * ts + ts / 2;
        spots = {{cx, door->y + door->h + ts / 2}, {cx, door->y - ts / 2}, {door->x + door->w + ts / 2, cy}, {door->x - ts / 2, cy}
    } else if (const MapObject* s = m.findByType("spawn", "start")) {
        spots = {{s->cx(), s->cy()}, {s->cx(), s->cy() + ts}, {s->cx(), s->cy() - ts}, {s->cx() + ts, s->cy()}, {s->cx() - ts, s->c
    } else {
        spots = {{ts * 1.5f, ts * 1.5f}};
    }
    for (const auto& sp : spots)
        if (!probe.collides(sp.first, sp.second, collision)) return sp;
    return spots.front();
}
```

Walk through `arrivalPoint`: after going through a door you should appear next to the matching door in the new room, not inside a wall. It finds the door object in the new room whose `to` property is the room you came from. It builds four candidate standing spots, snapped to tile centres: just below the door, above, to the right, to the left. The first one where the player’s box does not collide wins. If there is no matching door (the very first room) it uses the `start` spawn point, and as a last resort a fixed spot. The `probe` parameter is a copy of the player used only to test `collides`.

Interact.h, lines 69 to 87

```
inline std::vector<char> reachableTiles(const TileMap& m, const Player& probe, int sx, int sy) {
    const int w = m.width(), h = m.height(), ts = m.tileSize();
    std::vector<char> seen(static_cast<size_t>(w * h), 0);
    auto free = [&](int x, int y) {
        return x >= 0 && y >= 0 && x < w && y < h && !probe.collides((x + 0.5f) * ts, (y + 0.5f) * ts, m);
    };
    if (!free(sx, sy)) return seen;
    std::queue<std::pair<int, int>> q;
    q.push({sx, sy}); seen[static_cast<size_t>(sy * w + sx)] = 1;
    const int dx[4] = {1, -1, 0, 0}, dy[4] = {0, 0, 1, -1};
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int k = 0; k < 4; ++k) {
            int nx = x + dx[k], ny = y + dy[k];
            if (free(nx, ny) && !seen[static_cast<size_t>(ny * w + nx)]) { seen[static_cast<size_t>(ny * w + nx)] = 1; q.push({nx,
        }
    }
    return seen;
}
```

Walk through `reachableTiles`: a BFS on the tile grid. `free(x, y)` says a tile is inside the map and the player's box fits there. Starting from one tile it visits the four neighbours (right, left, down, up, from the `dx` and `dy` arrays), marking visited tiles in a flat array indexed by `y * w + x`. The tests use it to prove that every clue, door and person in every room can actually be walked to.

9. The screen loop and keys (GameWidget.cpp)

What it does: runs the game about 62 times a second, turns key presses into engine calls, and loads rooms.

GameWidget.cpp, lines 114 to 123

```
void GameWidget::tick() {
    float dt = std::min(clock_.restart() / 1000.0f, 0.05f);  // clamp so a lag spike can't teleport us through walls
    elapsed_ += dt;
    if (mode_ == Mode::Explore && !hasMessage()) {
        float dx = (down(Qt::Key_D, Qt::Key_Right) ? 1.f : 0.f) - (down(Qt::Key_A, Qt::Key_Left) ? 1.f : 0.f);
        float dy = (down(Qt::Key_S, Qt::Key_Down) ? 1.f : 0.f) - (down(Qt::Key_W, Qt::Key_Up) ? 1.f : 0.f);
        player_.move(dx, dy, dt, collision_);
    }
    update();
}
```

Walk through `tick`:

1. `clock_.restart()` returns the milliseconds since the last frame and starts timing again. Dividing by 1000 gives seconds, and `std::min(..., 0.05f)` caps it at 0.05 s. The cap means that after a pause (a window drag, a slow frame) the player cannot take one giant step through a wall.
2. `elapsed_` accumulates total time; the pulsing glints and the bobbing question marks use it with `sin`.
3. Only in Explore mode, with no message open, are the held keys read. `down(D, Right)` is true if either key is currently held; subtracting the left keys from the right keys gives -1, 0 or 1. The result goes to `player_.move(...)`.
4. `update()` asks Qt to call `paintEvent` soon, which redraws everything.

GameWidget.cpp, lines 181 to 212

```
void GameWidget::keyPressEvent(QKeyEvent* e) {
    const int k = e->key();
    if (e->isAutoRepeat() && k != Qt::Key_Up && k != Qt::Key_Down && k != Qt::Key_Backspace) return;
    keys_.insert(k);   // always remember held keys, even while a message or overlay is open
    switch (mode_) {
        case Mode::Dial: handleDialKey(e); return;
        case Mode::Accuse: handleAccuseKey(k); return;
        case Mode::Dossier: handleDossierKey(k); return;
        case Mode::Talk: handleTalkKey(k); return;
        case Mode::Present: handlePresentKey(k); return;
        case Mode::LabMenu: handleLabMenuKey(k); return;
        case Mode::LabTod: handleLabTodKey(k); return;
        case Mode::LabInk: handleLabInkKey(k); return;
        case Mode::Ending: if (k == Qt::Key_R) restart(); return;
        case Mode::GameOver: if (k == Qt::Key_R) restart(); return;
        case Mode::Explore: break;
    }
    if (hasMessage()) {
        const bool walkKey = k == Qt::Key_W || k == Qt::Key_A || k == Qt::Key_S || k == Qt::Key_D ||
                             k == Qt::Key_Left || k == Qt::Key_Right || k == Qt::Key_Up || k == Qt::Key_Down;
        if (walkKey || k == Qt::Key_E || k == Qt::Key_Space || k == Qt::Key_Return || k == Qt::Key_Enter || k == Qt::Key_Escape) {
            msgTitle_.clear();   // any walking key closes the message, and you keep walking
            msgBody_.clear();
        }
        return;
    }
    if (k == Qt::Key_E || k == Qt::Key_Space) interact();
    else if (k == Qt::Key_Tab || k == Qt::Key_J) { mode_ = Mode::Dossier; dossierPage_ = 0; scroll_ = 0; }
    else if (k == Qt::Key_V) { mode_ = Mode::Accuse; accuseIdx_ = 0; }
    else if (k == Qt::Key_L) { mode_ = Mode::LabMenu; labIdx_ = 0; labNote_.clear(); }
    else if (k == Qt::Key_F1) debug_ = !debug_;
}
```

Walk through `keyPressEvent`:

1. Auto-repeat (the key held down generating repeated presses) is ignored, except Up, Down and Backspace so menus can scroll quickly.
2. The key is always added to `keys_`, even inside menus, so `tick()` can poll it later.
3. `switch (mode_)` sends the key to the handler for the current screen and returns. This is the state machine: the same key means different things in Talk, Lab or Dossier mode. Only `Mode::Explore` falls through to the code below.
4. If a message box is open, a walking key, E, Space, Enter or Escape closes it. Walking keys still count, so the player keeps moving.
5. Otherwise the Explore keys: E or Space calls `interact()`; Tab or J opens the dossier; V opens the accusation list; L the lab; F1 toggles the debug view.

GameWidget.cpp, lines 145 to 172

```
void GameWidget::interact() {
    const MapObject* o = nearestInteractable(tiled_, *game_, player_.x, player_.y);
    if (!o) return;
    if (o->type == "door") {
        const Id from = game_->location();
        Result r = game_->move(o->prop("to"));
        if (r == Result::Ok) { loadLocation(game_->location(), from); return; }
        const std::string hint = o->prop("locked_hint");
        say("Locked", q(r == Result::Locked && !hint.empty() ? hint : resultMessage(r)));
    } else if (o->type == "clue") {
        const Id cid = o->prop("clue_id");
        Result r = game_->collect(cid);
        if (r == Result::Ok) { refreshGates(); showClue(cid); }
        else say(q(o->name), q(resultMessage(r)));
    } else if (o->type == "npc") {
        startTalk(*o);
    } else if (o->type == "dial") {
        const Clue* k = case_.clue(o->prop("clue_id"));
        if (k && !prerequisitesMet(*k, game_->heldSet())) {
            say("Dial lock", "An iron door with a brass dial. It wants a year in the English calendar. You have no idea which.");
        } else {
            dialObj_ = *o;
            dialInput_.clear();
            dialWrong_ = false;
            mode_ = Mode::Dial;
        }
    }
}
```

Walk through `interact`: ask for the nearest active object; do nothing if there is none. Then by type:

- **door:** remember the room you are leaving (`from`), ask the engine to `move`. If `Ok`, load the new room, passing `from` so the arrival point is next to the right door. If not, show the door's custom `locked_hint` when the result is `Locked`, otherwise a standard message.
- **clue:** ask the engine to `collect` it. On success open any gates it unlocks and show the clue text; otherwise show why not.
- **npc:** start a conversation.
- **dial:** if the dial's clue has unmet prerequisites (you do not hold the nameplate) show a hint; otherwise store the dial object and switch to Dial mode.

GameWidget.cpp, lines 82 to 90

```
void GameWidget::loadLocation(const Id& id, const Id& from) {
    tiled_ = TiledMap::fromFile(std::string(ASSET_DIR) + "/maps/" + id + ".json");
    collision_ = tiled_.collisionMap();
    for (const auto& w : checkMap(tiled_, case_, id)) qWarning().noquote() << "MAP WARNING (" << q(id) << "):" << q(w);
    auto pt = arrivalPoint(tiled_, collision_, from, player_);
    player_.x = pt.first;
    player_.y = pt.second;
    refreshGates();
}
```

GameWidget.cpp, lines 93 to 103

```
void GameWidget::refreshGates() {
    const int ts = collision_.tileSize();
    for (const auto& o : tiled_.objects()) {
        if (o.type != "dial") continue;
        const bool open = game_->has(o.prop("clue_id"));
        const int x0 = static_cast<int>(std::floor(o.x / ts)), x1 = static_cast<int>(std::ceil((o.x + o.w) / ts)) - 1;
        const int y0 = static_cast<int>(std::floor(o.y / ts)), y1 = static_cast<int>(std::ceil((o.y + o.h) / ts)) - 1;
        for (int ty = y0; ty <= y1; ++ty)
            for (int tx = x0; tx <= x1; ++tx) collision_.set(tx, ty, open ? '.' : '#');
    }
}
```

Walk through `loadLocation`: reads and parses the room's JSON file, builds the collision grid, runs `checkMap` and prints any problems as `MAP WARNING`, places the player at the arrival point, then calls `refreshGates`.

Walk through `refreshGates`: the dial lock is a door made of solid tiles. For every `dial` object, it works out which tile cells the object's rectangle covers (`floor` for the start, `ceil` minus one for the end) and sets them to `'.'` (open) if you hold the dial's clue, otherwise `'#'` (solid). It runs after any action that could earn that clue and every time a room loads.

Worked example: pressing E at the library door. You stand in the hall within 36 pixels of the "The library" door object. `nearestInteractable` returns it. `interact()` sees `type == "door"`, saves `from = "hall"`, calls `game_->move("library")`. The engine finds `library` in the hall's exits, sees no lock, pushes `Move(hall)` on the undo stack and sets the location. `interact()` calls `loadLocation("library", "hall")`: it loads `library.json`, builds the walls, finds the library's door whose `to` is `hall`, and places you just below it. Next frame the library is drawn.

10. Cross-examination on screen

What it does: the Talk screen browses a person's statements; pressing asks for detail; presenting shows evidence. The screen only keeps a few variables and calls `Game::press` and `Game::present`.

GameWidget.cpp, lines 263 to 285

```
        void GameWidget::handleTalkKey(int k) {
            const auto& st = game_->statements(talkSuspect_);
            const int n = static_cast<int>(st.size());
            auto show = [&](int idx) {
                talkIdx_ = idx;
                talkNote_.clear();
                if (game_->exposed(talkSuspect_, static_cast<size_t>(idx))) { talkLine_ = q(st[static_cast<size_t>(idx)].confession); talkT
"LIE EXPOSED"; }
                else { talkLine_ = q(st[static_cast<size_t>(idx)].text); talkTag_.clear(); }
            };
            if (k == Qt::Key_Escape) { mode_ = Mode::Explore; return; }
            if (k == Qt::Key_Left || k == Qt::Key_A) show((talkIdx_ + n - 1) % n);
            else if (k == Qt::Key_Right || k == Qt::Key_D) show((talkIdx_ + 1) % n);
            else if (k == Qt::Key_P) {
                PressOutcome o = game_->press(talkSuspect_, static_cast<size_t>(talkIdx_));
                talkLine_ = q(o.text);
                talkTag_ = "PRESSED";
                talkNote_ = o.revealed.empty() ? QString() : "New evidence: " + q(case_.clue(o.revealed)->name()) + " (press Tab to read it
            } else if (k == Qt::Key_T || k == Qt::Key_Return || k == Qt::Key_Enter) {
                if (game_->dossier().empty()) { talkNote_ = "You have no evidence to present yet."; return; }
                presentIdx_ = 0;
                mode_ = Mode::Present;
            }
        }
```

Walk through `handleTalkKey`:

- `st` is the list of the person's statements and `n` its size. The small lambda `show(idx)` sets which statement is displayed. If that statement is already exposed it shows the *confession* with the tag "LIE EXPOSED"; otherwise it shows what the person says.
- Left or A shows the previous statement, Right or D the next. `(talkIdx_ + n - 1) % n` and `(talkIdx_ + 1) % n` wrap around: after the last statement comes the first.
- **P (press):** asks the engine, shows the press text with the tag "PRESSED", and if a clue was earned, adds a note "New evidence: … (press Tab to read it)".
- **T or Enter (present):** if the dossier is empty it only shows a note. Otherwise it resets the evidence cursor and switches to `Mode::Present`.
- Escape returns to Explore.

GameWidget.cpp, lines 287 to 309

```
void GameWidget::handlePresentKey(int k) {
    const auto& list = game_->dossier();
    const int n = static_cast<int>(list.size());
    if (k == Qt::Key_Escape) { mode_ = Mode::Talk; return; }
    if (n == 0) { mode_ = Mode::Talk; return; }
    if (k == Qt::Key_Up) presentIdx_ = (presentIdx_ + n - 1) % n;
    else if (k == Qt::Key_Down) presentIdx_ = (presentIdx_ + 1) % n;
    else if (k == Qt::Key_Return || k == Qt::Key_Enter) {
        PresentOutcome o = game_->present(talkSuspect_, static_cast<size_t>(talkIdx_), list[static_cast<size_t>(presentIdx_)]);
        mode_ = Mode::Talk;
        if (o.kind == Present::Exposed) {
            talkLine_ = q(o.text);
            talkTag_ = "LIE EXPOSED";
            talkNote_ = o.revealed.empty() ? QString() : "New evidence: " + q(case_.clue(o.revealed)->name()) + " (press Tab to rea
            refreshGates();
        } else if (o.kind == Present::Wrong) {
            talkNote_ = "That does not contradict what they said. Credibility -1.";
            checkFailure();
        } else {
            talkNote_ = q(o.text);
        }
    }
}
```

Walk through `handlePresentKey`: `list` is the dossier in the order clues were found. Up and Down move the cursor (wrapping). On Enter the screen calls `game_->present(suspect, statement, chosen clue)`, goes back to Talk mode, and branches on the result:

- `Exposed`: show the confession, the tag and the new-evidence note, then `refreshGates()` because the new clue might open a gate.
- `Wrong`: show “Credibility -1” and call `checkFailure()`, which switches to Game Over if credibility reached 0.
- anything else (`AlreadyExposed` and so on): show the engine’s text.

GameWidget.cpp, lines 222 to 239

```
void GameWidget::handleAccuseKey(int k) {
    const int n = static_cast<int>(accusable_.size());
    if (k == Qt::Key_Escape || k == Qt::Key_V) { mode_ = Mode::Explore; return; }
    if (k == Qt::Key_Up) accuseIdx_ = (accuseIdx_ + n - 1) % n;
    else if (k == Qt::Key_Down) accuseIdx_ = (accuseIdx_ + 1) % n;
    else if (k == Qt::Key_Return || k == Qt::Key_Enter) {
        const Suspect* s = case_.suspect(accusable_[static_cast<size_t>(accuseIdx_)]);
        mode_ = Mode::Explore;
        if (game_->accuse(s->id()) == Verdict::Solved) { mode_ = Mode::Ending; return; }
        // Wrong suspect and "right suspect, not enough exposed lies" get the SAME answer, so guessing teaches nothing,
        // and a wrong accusation costs credibility.
        game_->penalize(2);
        say("Not enough proof (credibility -2)",
            "You accuse " + q(s->name()) + ", but the case will not hold up. Keep cross-examining people: catch the real "
            "liar with evidence that contradicts what they say.");
        checkFailure();
    }
}
```

Walk through `handleAccuseKey`: `accusable_` (built in the constructor) lists everyone who can be accused. Up and Down move with wrap-around. On Enter, ask the engine: if `Solved`, switch to Ending. For any other verdict the screen applies `penalize(2)` and shows one message (“Not enough proof”) for both “wrong person” and “right person without proof”, so guessing teaches nothing. Then `checkFailure()` in case credibility hit 0.

11. Drawing the world

What it does: every frame, `paintEvent` fills the background, applies the camera, draws the room and the people, then draws the interface on top.

GameWidget.cpp, lines 847 to 858

```
        const float mapW = static_cast<float>(tiled_.width() * tiled_.tileW());
        const float mapH = static_cast<float>(tiled_.height() * tiled_.tileH());
        // Zoom in so the art is readable; the camera follows the player and stops at the map edges.
        const float fit = std::min(width() / mapW, height() / mapH);
        const float sc = std::max(fit, 2.0f);
        float offX = (width() - mapW * sc) / 2, offY = (height() - mapH * sc) / 2;   // map smaller than window: centre it
        if (mapW * sc > width()) offX = std::min(0.f, std::max(width() - mapW * sc, width() / 2.f - player_.x * sc));
        if (mapH * sc > height()) offY = std::min(0.f, std::max(height() - mapH * sc, height() / 2.f - player_.y * sc));
        p.translate(offX, offY);
        p.scale(sc, sc);
        drawWorld(p);
        p.restore();
```

Walk through the camera:

- `mapW`, `mapH` are the room's size in pixels (tiles times tile size).
- `fit` is the largest scale at which the whole room fits the window; `sc = max(fit, 2.0)` forces at least 2x zoom so the pixel art is readable, even if that means the room no longer fits.
- `offX, offY` is how far the picture is shifted. If the scaled room is smaller than the window, it is centred. If it is larger, the shift tries to keep the player at the centre of the screen (`width/2 - player.x * sc`), but is clamped between "room's right edge touches the window's right edge" and 0 ("room's left edge touches the window's left edge"), so the view never shows empty space outside the map.
- `translate` then `scale` apply the shift and zoom to everything `drawWorld` draws; `restore` removes them so the interface is drawn at normal size.

Worked example (hall). Window 960 x 720, hall 24 x 18 tiles = 768 x 576 pixels. `fit = min(960/768, 720/576) = 1.25`, so `sc = 2`. The scaled room is 1536 wide, wider than the window, so the camera follows. Player at x = 384: `480 - 384*2 = -288`, within the allowed range -576 to 0, so the shift is -288 and the player appears at screen x = 384*2 - 288 = 480, the centre. Player at x = 50: `480 - 100 = 380`, above the maximum 0, so the shift is clamped to 0 and the camera stops at the left wall. Player at x = 700: `480 - 1400 = -920`, below the minimum -576, so it is clamped to -576 and the right wall touches the window edge.

GameWidget.cpp, lines 372 to 379

```
        void GameWidget::drawCharacter(QPainter& p, int sprite, int dir, int frame, float cx, float feetY) {
            const int fw = 64, fh = 96;
            QRectF src((dir * 4 + frame) * fw, sprite * fh, fw, fh);
            QRectF dst(cx - 16.0, feetY - 45.0, 32.0, 48.0);
            p.setRenderHint(QPainter::SmoothPixmapTransform, true);
            p.drawPixmap(dst, characters_, src);
            p.setRenderHint(QPainter::SmoothPixmapTransform, false);
        }
```

Walk through `drawCharacter`: the sprite sheet has one row per character (64 x 96 pixel frames) and 16 columns: 4 directions times 4 walk frames. `src` selects the frame at x = `(dir * 4 + frame) * 64`, y = `sprite * 96`. `dst` is drawn at 32 x 48 centred on `cx` with the feet at `feetY` (about 45 pixels above `feetY` for the head). Smooth scaling is switched on only for this call, because the characters are downscaled from 64 to 32 and look better smoothed, while the tiles are kept crisp.

Worked example. Raghav is sprite 1. Facing left is direction 1, frame 2: `x = (1*4 + 2) * 64 = 384`, `y = 1 * 96 = 96`, a 64 x 96 rectangle at (384, 96) of the sheet.

GameWidget.cpp, lines 406 to 425

```
        // people, drawn back-to-front so whoever is lower on the screen appears in front
        struct Actor { float feetY, cx; int sprite, dir, frame; bool pending; };
        std::vector<Actor> actors;
        for (const auto& o : tiled_.objects()) {
            if (o.type != "npc") continue;
            std::string sp = o.prop("sprite");
            if (sp.empty()) sp = "raghav";
            const float cx = o.cx(), feet = o.cy() + 9.f;
            int dir = 0;                                  // people turn to face you when you come close
            const float dx = player_.x - cx, dy = player_.y - feet;
            if (std::sqrt(dx * dx + dy * dy) < 150.f) dir = std::fabs(dx) > std::fabs(dy) ? (dx < 0 ? 1 : 2) : (dy < 0 ? 3 : 0);
            const Id sid = o.prop("suspect_id");
            bool pending = false;                         // a small "!" over people who still have a lie you can catch
            for (size_t i = 0; i < game_->statements(sid).size(); ++i)
                if (game_->statements(sid)[i].isLie() && !game_->exposed(sid, i)) pending = true;
            actors.push_back({feet, cx, spriteIndex(sp), dir, 0, pending});
        }
        const int walkFrame = player_.moving ? static_cast<int>(player_.anim * 9.f) % 4 : 0;
        actors.push_back({player_.y + player_.half, player_.x, 0, player_.facing, walkFrame, false});
        std::sort(actors.begin(), actors.end(), [](const Actor& a, const Actor& b) { return a.feetY < b.feetY; });
```

Walk through the people code: for each `npc` object, the position is the object's centre, with the feet 9 pixels below it. If the player is within 150 pixels, the person turns to face the player (the larger of the horizontal and vertical difference picks left or right versus up or down). A small "?" is shown above anyone who still has a lie that is not yet exposed (a loop over their statements using `isLie()` and `exposed(...)`). The player is added to the same list, with its walk frame computed as `int(anim * 9) % 4` (nine frames per second cycling through four pictures). Finally `std::sort` orders by feet y, so a character lower on the screen is drawn later and therefore in front. This is the painter's algorithm.

12. The lab and dial screens

What it does: the time-of-death screen lets you set hours and minutes with the arrow keys; the dial screen takes four digits; the ink screen takes a key from 1 to 3. Each compares your answer with the pure maths in Forensics.h or with the code stored in the map.

GameWidget.cpp, lines 329 to 343

```
        void GameWidget::handleLabTodKey(int k) {
            if (k == Qt::Key_Escape) { mode_ = Mode::LabMenu; return; }
            if (k == Qt::Key_Left || k == Qt::Key_Right) todField_ = 1 - todField_;
            else if (k == Qt::Key_Up) { if (todField_ == 0) todHour_ = (todHour_ + 1) % 24; else todMinute_ = (todMinute_ + 5) % 60; }
            else if (k == Qt::Key_Down) { if (todField_ == 0) todHour_ = (todHour_ + 23) % 24; else todMinute_ = (todMinute_ + 55) % 60; }
            else if (k == Qt::Key_Return || k == Qt::Key_Enter) {
                if (todCorrect(tod_, todHour_ * 60 + todMinute_)) {
                    game_->completeLab("lab_tod");
                    mode_ = Mode::Explore;
                    showClue("c_tod");
                } else {
                    labNote_ = "That does not fit the numbers. How many hours does it take to cool from 37 to 29 degrees? Count back from 7
am.";
                }
            }
        }
```

Walk through `handleLabTodKey`: Left or Right switches between the hour field (0) and minute field (1) with `1 - todField_`. Up and Down change the active field with wrap-around: hours `(h + 1) % 24` and `(h + 23) % 24` (which is minus one); minutes in steps of five, `(m + 5) % 60` and `(m + 55) % 60`. Enter converts the choice to minutes after midnight, `todHour_ * 60 + todMinute_`, and asks `todCorrect`. On success the engine completes the lab (earning `c_tod`) and the clue text is shown; on failure a hint is shown.

GameWidget.cpp, lines 345 to 356

```
void GameWidget::handleLabInkKey(int k) {
    if (k == Qt::Key_Escape) { mode_ = Mode::LabMenu; return; }
    if (k >= Qt::Key_1 && k <= Qt::Key_3) {
        if (k - Qt::Key_1 == inkMatch(ink_)) {
            game_->completeLab("lab_ink");
            mode_ = Mode::Explore;
            showClue("c_ink_match");
        } else {
            labNote_ = "These Rf values do not match the forged ink. Work each one out: spot distance divided by solvent distance."
        }
    }
}
```

Walk through `handleLabInkKey`: keys 1 to 3 choose a pen. `k - Qt::Key_1` converts the key to a zero-based index and it is compared with `inkMatch(ink_)`, which is 2 for Raghav's pen. So key 3 is the right answer.

GameWidget.cpp, lines 241 to 260

```
void GameWidget::handleDialKey(QKeyEvent* e) {
    const int k = e->key();
    if (k == Qt::Key_Escape) { mode_ = Mode::Explore; return; }
    if (k == Qt::Key_Backspace) { dialInput_.chop(1); dialWrong_ = false; return; }
    if (k >= Qt::Key_0 && k <= Qt::Key_9) {
        if (dialInput_.size() < 4) dialInput_ += QChar('0' + (k - Qt::Key_0));
        dialWrong_ = false;
        return;
    }
    if ((k == Qt::Key_Return || k == Qt::Key_Enter) && dialInput_.size() == 4) {
        if (dialInput_ == q(dialObj_.prop("code"))) {
            const Id cid = dialObj_.prop("clue_id");
            mode_ = Mode::Explore;
            if (game_->collect(cid) == Result::Ok) { refreshGates(); showClue(cid); }
        } else {
            dialWrong_ = true;
            dialInput_.clear();
        }
    }
}
```

Walk through `handleDialKey`: Escape leaves; Backspace removes the last digit (`chop(1)`); keys 0 to 9 append a digit while fewer than four are typed; Enter with four digits compares the text with the map object's `code` property ("1921"). If it matches, the engine collects the dial clue, `refreshGates()` opens the solid door tiles and the clue text is shown. A wrong code clears the input and sets `dialWrong_` so the screen can show "wrong".

13. How the tests prove things

tests.cpp, lines 20 to 21

```
static int failures = 0;
#define CHECK(cond) do { if (!(cond)) { std::cerr << "FAIL line " << __LINE__ << ": " #cond "\n"; failures++; } } while (0)
```

Walk through the macro: `CHECK(cond)` evaluates the condition; if it is false it prints the line number (`__LINE__`) and the condition as text (`#cond` turns the argument into a string) and increments `failures`. At the end `main` returns an error code if any check failed. The `do { ... } while (0)` wrapper makes the macro behave like one statement.

tests.cpp, lines 188 to 198

```
        // Round 2: cross-examine. A wrong piece of evidence costs credibility.
        CHECK(rg.present("s_rohit", 0, "c_scene").kind == Present::Wrong);
        CHECK(rg.credibility() == 5);
        CHECK(rg.present("s_rohit", 0, "c_betting").kind == Present::NotHeld);   // you can't present what you don't have
        CHECK(rg.credibility() == 5);
        auto rohit = rg.present("s_rohit", 0, "c_mill_report");
        CHECK(rohit.kind == Present::Exposed && rohit.revealed == "c_rohit_testimony");
        CHECK(rg.present("s_rohit", 0, "c_mill_report").kind == Present::AlreadyExposed);
        CHECK(rg.exposedLies("s_rohit") == 1 && rg.exposedIncriminating("s_rohit") == 0);
        CHECK(rg.present("s_rohit", 1, "c_scene").kind == Present::Wrong);       // a TRUE statement cannot be contradicted
        CHECK(rg.credibility() == 4);
```

Walk through this excerpt of the scripted playthrough: `rg` is a real `Game` on the real Rastogi case. Presenting the wrong clue gives `Present::Wrong` and credibility drops from 6 to 5. Presenting a clue you do not hold is `NotHeld` and credibility stays 5. Presenting the right clue (`c_mill_report`) is `Exposed` and `revealed == "c_rohit_testimony"`. Presenting it again is `AlreadyExposed`. `exposedLies` is 1 but `exposedIncriminating` is 0, because Rohit's lie is not about the murder. Presenting a clue against his true statement is `Wrong` and credibility drops to 4. This is a test written as a story, and it checks exactly the engine rules from section 3.

tests.cpp, lines 305 to 317

```
        std::vector<int> path;
        for (int c2 = goal; c2 != -1; c2 = parent[static_cast<size_t>(c2)]) path.push_back(c2);
        std::reverse(path.begin(), path.end());
        Player walker = probe; walker.x = pt.first; walker.y = pt.second;
        bool ok2 = true;
        for (int cell : path) {
            const float tcx = (cell % W + 0.5f) * 32, tcy = (cell / W + 0.5f) * 32;
            int it = 0;
            while ((std::fabs(tcx - walker.x) > 2 || std::fabs(tcy - walker.y) > 2) && it < 600) { walker.move(tcx - walker.x, tcy - walker
0.016f, col); ++it; }
            if (it >= 600) { ok2 = false; break; }
        }
        if (!ok2) std::cerr << "STUCK walking to " << o.name << " in " << id << "\n";
        CHECK(ok2);
```

Walk through the walking simulation: earlier in the test, a BFS from the arrival point recorded for every reachable tile which tile it came from (`parent`). `path` follows those parent links back from the goal tile next to an object and reverses them into a route. A copy of the player (`walker`) then really walks that route using the real `Player::move` and the real collision grid, one frame at a time (`0.016` s per step), aiming at the centre of each tile on the route. If it cannot reach a tile within 600 frames it counts as stuck and the test fails. So the test proves that no clue, door or person in any of the 7 rooms is walled off, using the same movement and collision code the game uses.

14. One action through every layer

You show the mill report to Rohit. This is the full life of one action, from the keyboard to the pixels, naming every function that runs.

1. **Idle frames.** `tick()` runs every 16 ms. `nearestInteractable(...)` finds Rohit within 36 pixels, and `drawHud` shows "E Talk to: Rohit Rastogi".
2. **Press E.** Qt calls `keyPressEvent`. The mode is Explore and no message is open, so `interact()` runs; the nearest object has type `npc`, so `startTalk(o)` runs.
3. **`startTalk`** looks up the suspect from the object's `suspect_id` property, checks they have statements, remembers who we are talking to (`talkSuspect_`), picks the sprite, sets `talkIdx_ = 0`, loads the first statement's text and sets `mode_ = Mode::Talk`.
4. **Next frame.** `paintEvent` still draws the room (`drawWorld`) and then the overlay for `Mode::Talk` (`drawTalk`): Rohit's first statement, "After dinner I went to my room and stayed there…".
5. **Press T.** `keyPressEvent` sends the key to `handleTalkKey` (because of the mode switch). The dossier is not empty, so `presentIdx_ = 0` and `mode_ = Mode::Present`.
6. **Choose the clue.** Down moves `presentIdx_` through the dossier (with wrap-around). The cursor is on `c_mill_report`.
7. **Press Enter.** `handlePresentKey` calls `game_->present("s_rohit", 0, "c_mill_report")`.
8. **Inside the engine (`Game::present`).** The clue is held. The statement is not already exposed. `std::find` locates `c_mill_report` in `contradictedBy`. So `"s_rohit#0"` is inserted into `exposed_`, the confession text is put in the outcome, and `grant("c_rohit_testimony")` runs: the clue exists, is not held, its prerequisites are met, so `add` inserts it into `held_` and `found_` and pushes `Collect` on the undo stack. The outcome says `Exposed` with `revealed = "c_rohit_testimony"`.
9. **Back on the screen.** `mode_` returns to Talk. Because the kind is `Exposed`, `talkLine_` becomes the confession, `talkTag_` becomes "LIE EXPOSED", `talkNote_` says "New evidence: Rohit's confession (press Tab to read it)", and `refreshGates()` re-checks the dial gate.
10. **Next frame.** `drawTalk` shows the confession with the tag and note. In `drawWorld`, the "?" over Rohit disappears, because his only lie is now

exposed. The HUD goal (`currentObjective`) is recomputed from the new clue set. Tab now lists the new clue in the dossier.

The same function chain, screen to engine to data and back, handles every other action in the game, with different handlers.

15. One-page cheat sheet

Function	File	What it does in one line
`main`	main_gui.cpp	Starts Qt, creates the window, shows an error box if loading fails
`GameWidget::GameWidget`	GameWidget.cpp	Builds the case, the Game, loads pictures, starts the 16 ms timer
`makeRastogiCase`	CaseRastogi.h	Creates all rooms, clues, people, statements and labs
`Case::find`	Case.h	Template lookup of an id in any of the three hash maps
`Game::move`	Game.h	Checks exists, adjacent, not locked, then changes room and records undo
`Game::collect`	Game.h	Picks up a clue lying in this room if its prerequisites are held
`Game::grant`	Game.h	Earns a clue from a conversation or lab (no room needed)
`Game::add`	Game.h	The only way a clue enters the dossier; records it for undo
`Game::undo`	Game.h	Pops the stack and reverses a move or a pick-up
`Game::press`	Game.h	Shows extra detail and may earn a clue; free
`Game::present`	Game.h	Right evidence exposes a lie; wrong evidence costs 1 credibility
`Game::accuse`	Game.h	Solved only for the killer with 2 or more incriminating lies exposed
`reachableLocations`	Validator.h	BFS: which rooms can be entered with the clues held
`validate` (step 2)	Validator.h	Kahn's algorithm: clue prerequisites must have no cycle
`validate` (step 3)	Validator.h	Perfect player sweeps until every clue is obtained
`todAnswerMinute`, `todCorrect`	Forensics.h	Body-cooling maths and the 45-minute tolerance on a 24-hour circle
`inkMatch`	Forensics.h	Finds the single pen whose Rf matches the forged ink
`Player::move`	Player.h	Normalised, axis-separated movement with wall sliding
`Player::collides`	Player.h	Tests the four corners of the 18 by 18 box against walls
`Json::value`, `object`, `array`	JsonLite.h	Recursive-descent JSON parser producing a tree
`TiledMap::fromString`	TiledMap.h	Reads layers, objects, tileset and builds the collision grid
`objectActive`	Interact.h	Hidden objects wait for their clue; collected clues disappear
`nearestInteractable`	Interact.h	Closest usable object within 36 pixels
`arrivalPoint`	Interact.h	Where the player stands after a door (below, above, right, left)
`reachableTiles`	Interact.h	BFS over free tiles; used to prove nothing is walled off
`checkMap`	MapCheck.h	Cross-checks a map against the case and returns warnings
`GameWidget::tick`	GameWidget.cpp	Delta time, held-key movement, requests a repaint
`GameWidget::keyPressEvent`	GameWidget.cpp	Sends each key to the handler for the current mode
`GameWidget::interact`	GameWidget.cpp	What E does for doors, clues, people and the dial
`GameWidget::loadLocation`	GameWidget.cpp	Loads a room's map, walls, arrival point and gates
`GameWidget::paintEvent`	GameWidget.cpp	Camera, world, HUD, then the overlay for the current mode

Ten things to be able to explain without notes. (1) The three layers and why the engine has no Qt. (2) How `move` decides Locked. (3) The difference between `collect` and `grant`. (4) How `present` decides Exposed or Wrong. (5) What BFS does in `reachableLocations`. (6) How Kahn's algorithm detects a cycle. (7) How the perfect-player simulation ends. (8) How collision lets you slide along walls. (9) How the camera follows the player and stops at the edges. (10) The ten-step journey of showing the mill report to Rohit.